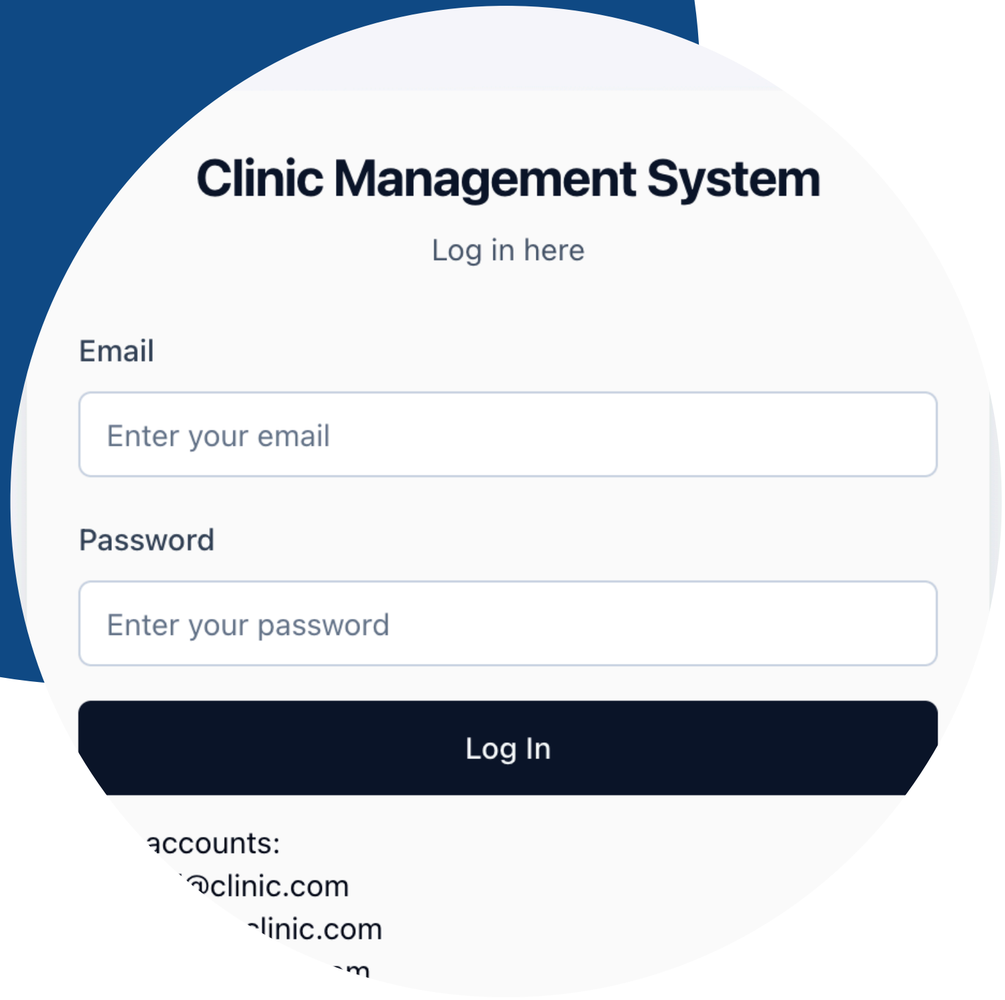
HEALTH CLINIC



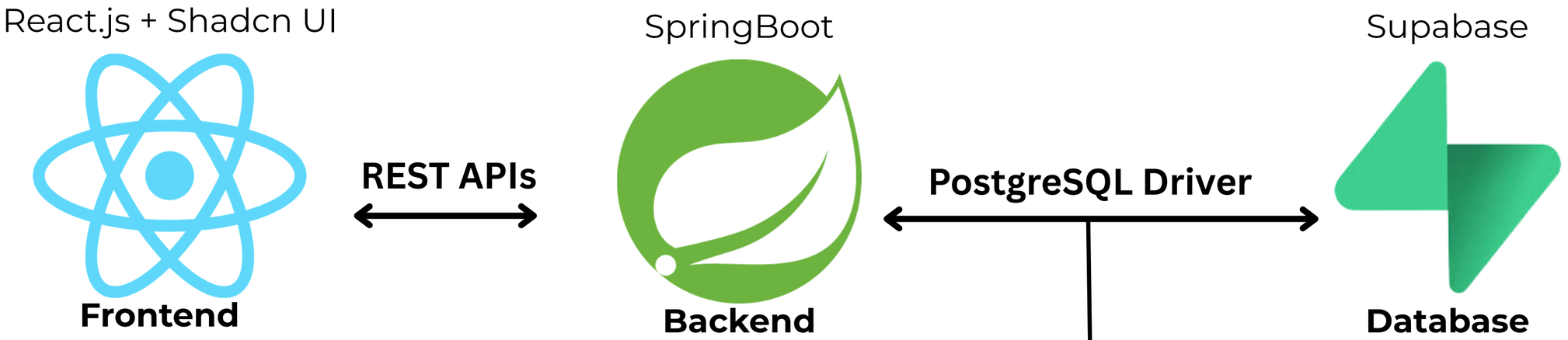
CONTENT

1. Workload division
2. System architecture
3. Sequence diagram
4. Class diagram
5. Design principles
6. Demo

WORKLOAD DIVISION

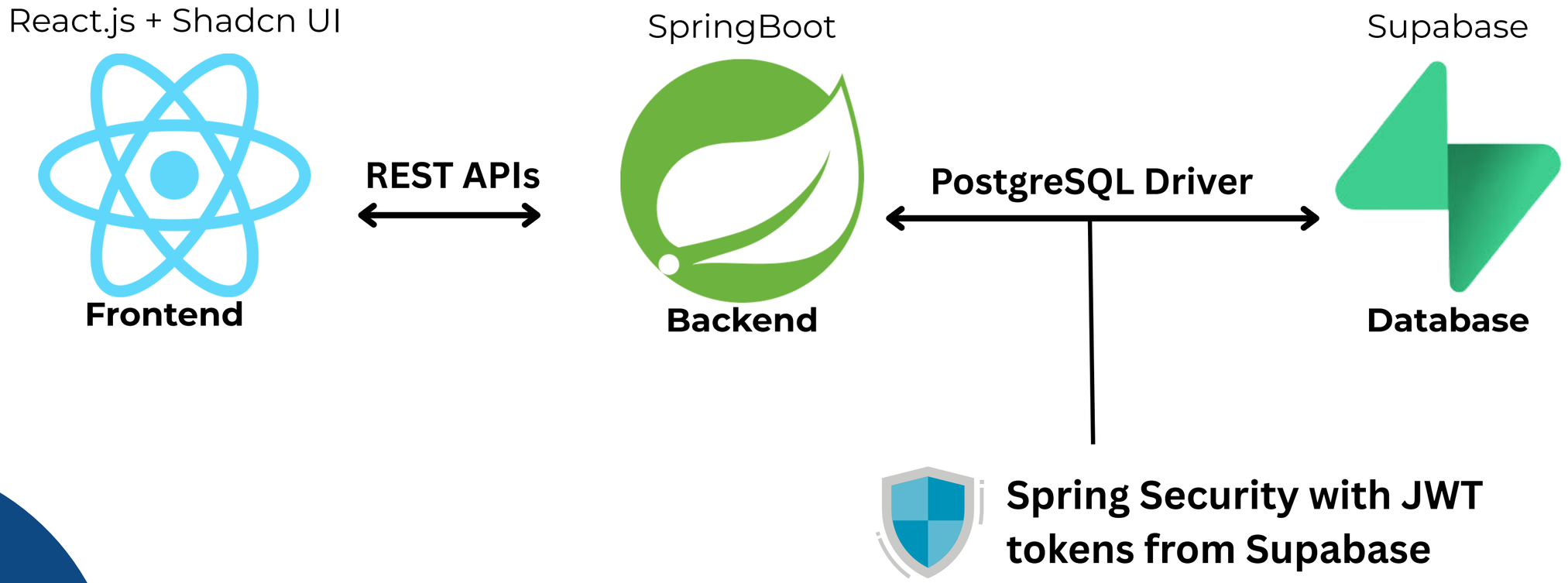
Member	Workload
Curteis	Project Setup, DB Connection, JWT Management
Jenis	Queue & QueueTicket backend, Patient Queue FE
Jia Wen	Notifications, Email integration, FE (Queue+Appt)
Joelyn	Appointment backend, Profile frontend and Display of appointments and queue for Staff (frontend)
Mynchin	Admin Management, Staff Walk-in Booking + BE
Zhe qing	Patient and Clinic BE, Patient Booking and Landing FE

SYSTEM ARCHITECURE



- JDBC Connection for database interaction
- Hikari Connection Pool for maintaining the pool of DB connections
- Hibernate for ORM mapping of the Java objects to DB tables

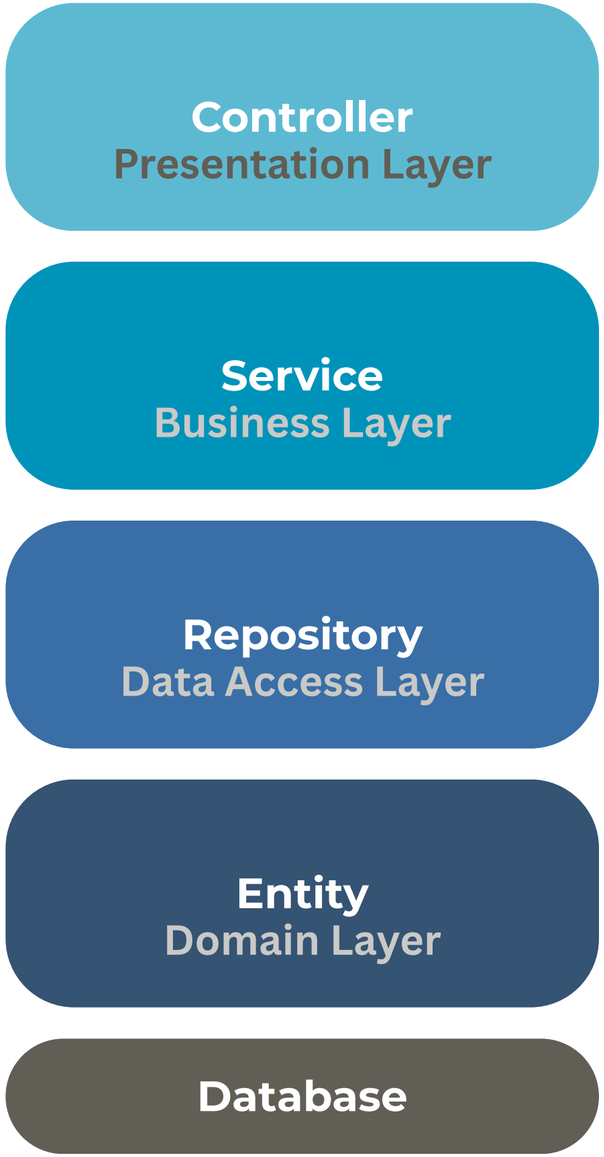
SYSTEM ARCHITECURE



LAYERED ARCHITECTURE (BACKEND)

SpringBoot Application Structure: 4 Main Layers

- ✅ Clear separation of concerns
- ✅ Modularity
- ✅ Scalability
- ✅ Simpler Maintenance
- ✅ Decoupling of layers



OPEN SOURCE LIBRARIES



Lombok
Reduces boilerplate code (@*Getters*, @*Setters*, @*AllArgsConstructor*)
Significantly reduces code verbosity, improves maintanability



Jackson
JSON processing

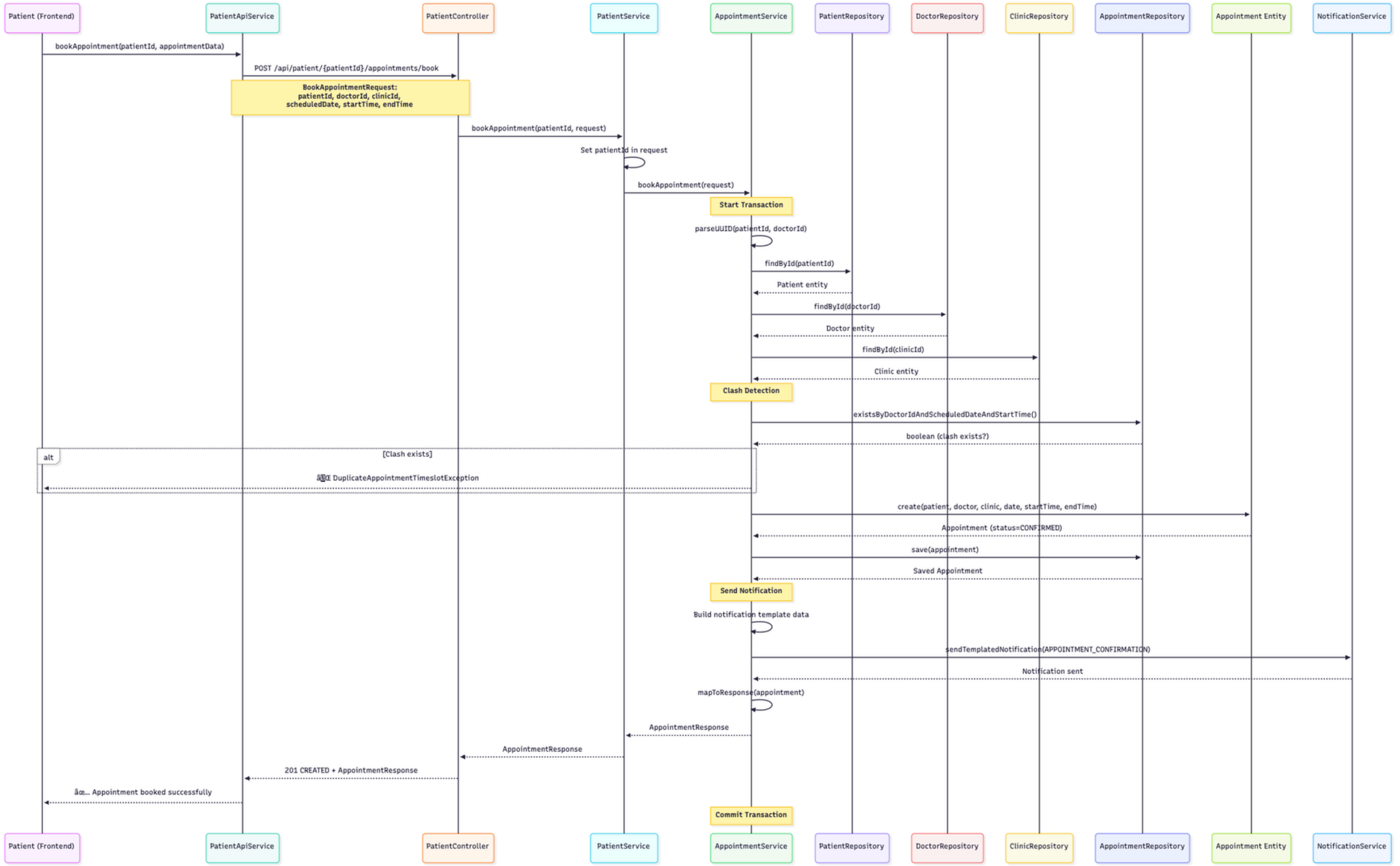
JPA + Hibernate vs **JDBC**
ORM with repository access and query derivation
Automatic Query generation from method names, reduces SQL boilerplate



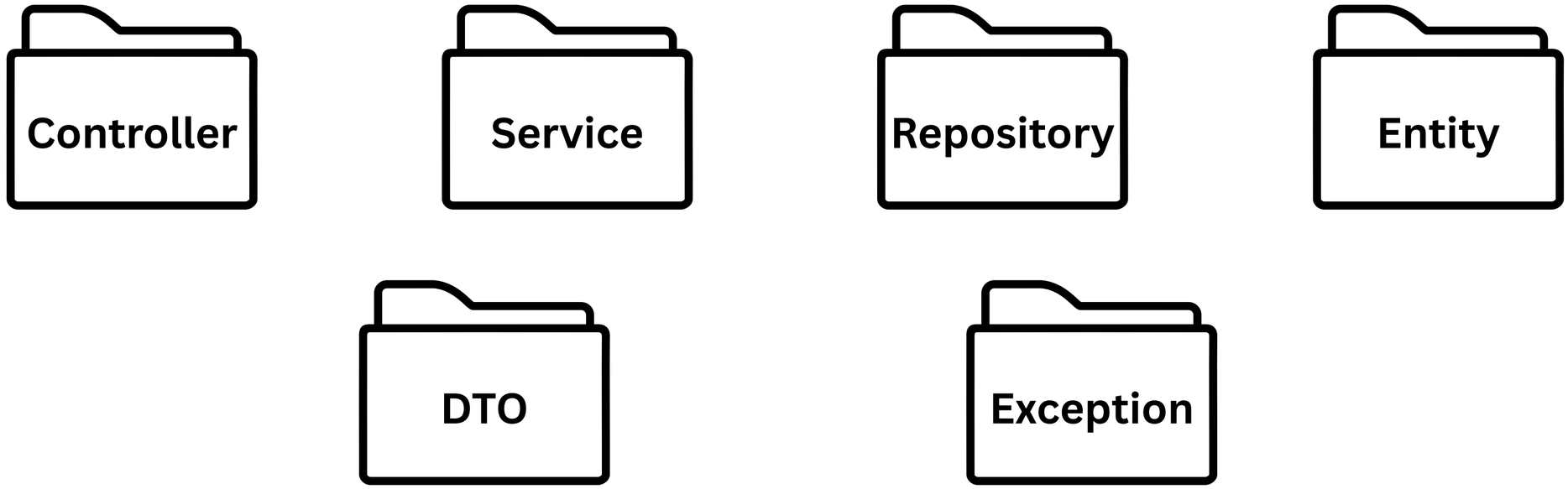
Spring Security
Authentication framework with seamless integration with Spring Boot

Spring Websocket vs **HTTP polling**
Efficient, lightweight, bidirectional connection between client and server, with seamless integration with Spring Boot

SEQUENCE DIAGRAM (Book Appointment)



CLASS DIAGRAM



DESIGN PRINCIPLES (SOLID)

Single Responsibility Principle (SRP)

Open Closed Principle (OCP)

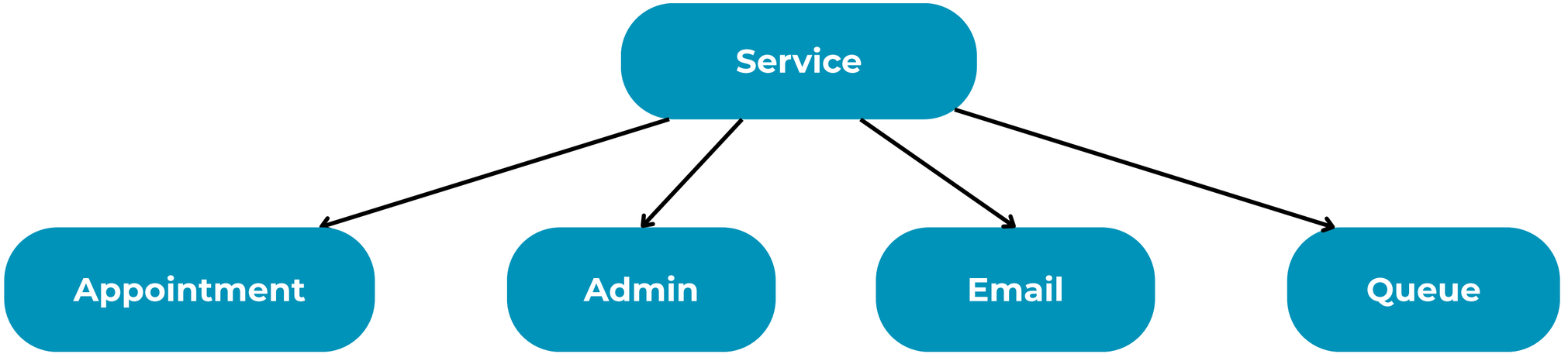
Liskov Substitution Principle (LSP)

Interface Segregration Principle (ISP)

Dependency Inversion Principle (DIP)

SINGLE RESPONSIBILITY PRINCIPLE (SRP)

Each service component has only **one** purpose



- Only manages appointment lifecycle
- Delegates to NotificationService for notifications
- Delegates to QueueTicketService for check-ins

- Create, update, delete persons
- Delegates to SupabaseAuthService for authentication

- Single purpose: Send emails via JavaMailSender
- No business logic, no template processing, no persistence

- Create, pause, resume, close queues
- Call next patient
- Delegates to NotificationService for notifications

OPEN CLOSED PRINCIPLE (OCP)

The Hierarchy (Categories)

```
ClinicException (base)
├── ValidationException → HTTP 400
├── ResourceException → HTTP 404
├── PolicyException → HTTP 422
└── SystemException → HTTP 500
```

- Categories are open for extension

The Handler (Polymorphic Catch)

- One handler per category catches all subclass exceptions
- No modification needed when adding new exceptions

OPEN CLOSED PRINCIPLE (OCP)

The Policy contract (Interface)

```java
public interface RescheduleCancelPolicy {

    /**
     * Check if an appointment can be rescheduled at the given time.
     *
     * @param appointment the appointment to reschedule
     * @param now current time
     * @return true if rescheduling is allowed
     */
    boolean canReschedule(Appointment appointment, Instant now);

    /**
     * Check if an appointment can be cancelled at the given time.
     *
     * @param appointment the appointment to cancel
     * @param now current time
     * @return true if cancellation is allowed
     */
    boolean canCancel(Appointment appointment, Instant now);
```

- Defines a stable, unchanging contract for behavior
- Acts as a boundary between business logic and policy rules

The Implementation

```java
public class DefaultRescheduleCancelPolicy implements RescheduleCancelPolicy {

    private static final long MIN_HOURS_BEFORE_MODIFY = 24L;
    private static final ZoneId CLINIC_ZONE = ZoneId.systemDefault();

    @Override
    public boolean canReschedule(Appointment appointment, Instant now) {
        // Cannot reschedule if already checked in or cancelled
        if (appointment.getStatus() == AppointmentStatus.CANCELLED
                || appointment.getStatus() == AppointmentStatus.CHECKED_IN) {
            return false;
        }

        // Must be at least 24 hours in advance
        return isAtLeast24HoursInAdvance(appointment, now);
    }

    @Override
    public boolean canCancel(Appointment appointment, Instant now) {
        // Cannot cancel if already checked in or cancelled
        if (appointment.getStatus() == AppointmentStatus.CANCELLED
                || appointment.getStatus() == AppointmentStatus.CHECKED_IN) {
            return false;
        }
```

- Multiple implementations of the same interface
- Each provides different business rules
- Can be swapped without code changes

The Consumer (Service/Entity)

```java
private final RescheduleCancelPolicy policy;
```

- Depends only on the interface, not concrete implementations

```java
public void cancel(UUID appointmentId) {
        Appointment appt = appointmentRepo.findById(appointmentId)
                .orElseThrow(() -> new EntityNotFoundException(
                        "Appointment not found: " + appointmentId));

        boolean ok = appt.cancel(policy, Instant.now());
```

- Never changes when new policies are added
- Delegates decisions to the injected policy

LISKOV SUBSTITUTION PRINCIPLE (LSP)

- Entity inheritance hierarchy ensures substitutability
  - All child classes can substitute their parent without breaking functionality

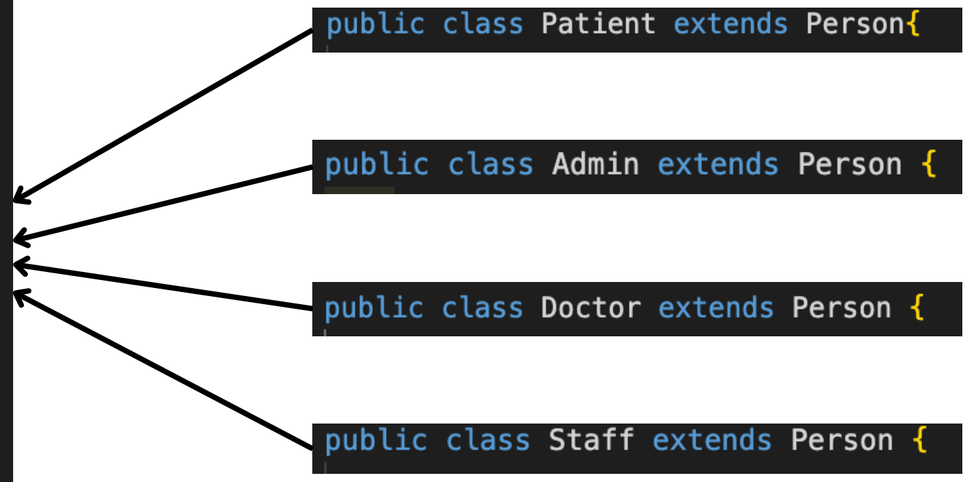
```java
public abstract class Person extends BaseEntity {

    @Column(name = "full_name")
    private String full_name;

    @Column(name = "email")
    private String email;

    @Column(name = "mobile")
    private String mobile;

    @Column(name = "role", columnDefinition = "ROLE")
    @Enumerated(EnumType.STRING)
    @JdbcTypeCode(SqlTypes.NAMED_ENUM)
    private Role role;
```



```java
public class Patient extends Person{
```

```java
public class Admin extends Person {
```

```java
public class Doctor extends Person {
```

```java
public class Staff extends Person {
```

  - Any method accepting Person can work with any subtype

- Role-specific behavior is added, not replaced (no overridden methods that change contract)

LISKOV SUBSTITUTION PRINCIPLE (LSP)

- Proper use of abstract classes
  - Forces concrete implementations to honor the contract

```java
public abstract class BaseEntity {

    @Id
    @Column(name = "id")
    private UUID id;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @UpdateTimestamp
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    // Generate UUID only if not already set (for entities that need auto-generation)
    @jakarta.persistence.PrePersist
    public void prePersist() {
        if (this.id == null) {
            UUID newId = UUID.randomUUID();
            this.id = newId;
            System.out.println("Generated new ID: " + newId);
        } else {
            System.out.println("ID already set, keeping: " + this.id);
        }
    }
}
```

```java
public abstract class Person extends BaseEntity {

    @Column(name = "full_name")
    private String full_name;

    @Column(name = "email")
    private String email;

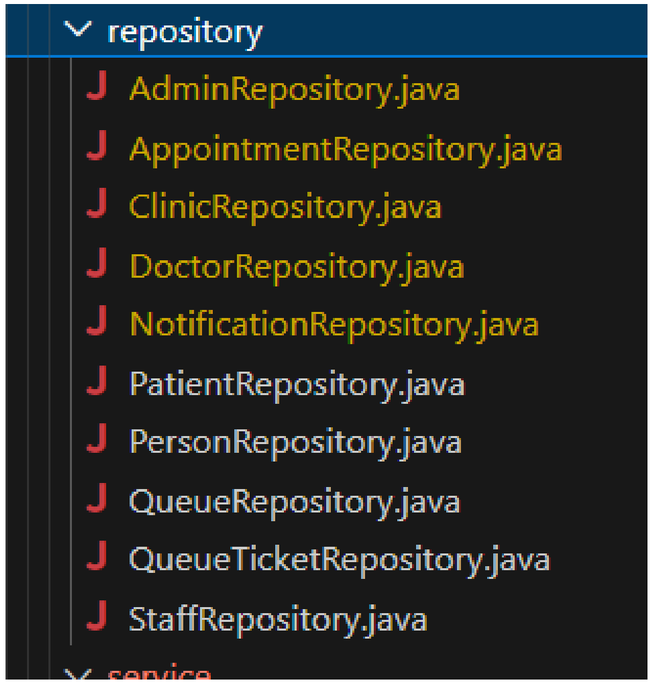
    @Column(name = "mobile")
    private String mobile;

    @Column(name = "role", columnDefinition = "ROLE")
    @Enumerated(EnumType.STRING)
    @JdbcTypeCode(SqlTypes.NAMED_ENUM)
    private Role role;

    // updateContactInfo() and getFullName() are settled by @Getter and @Setter
}
```

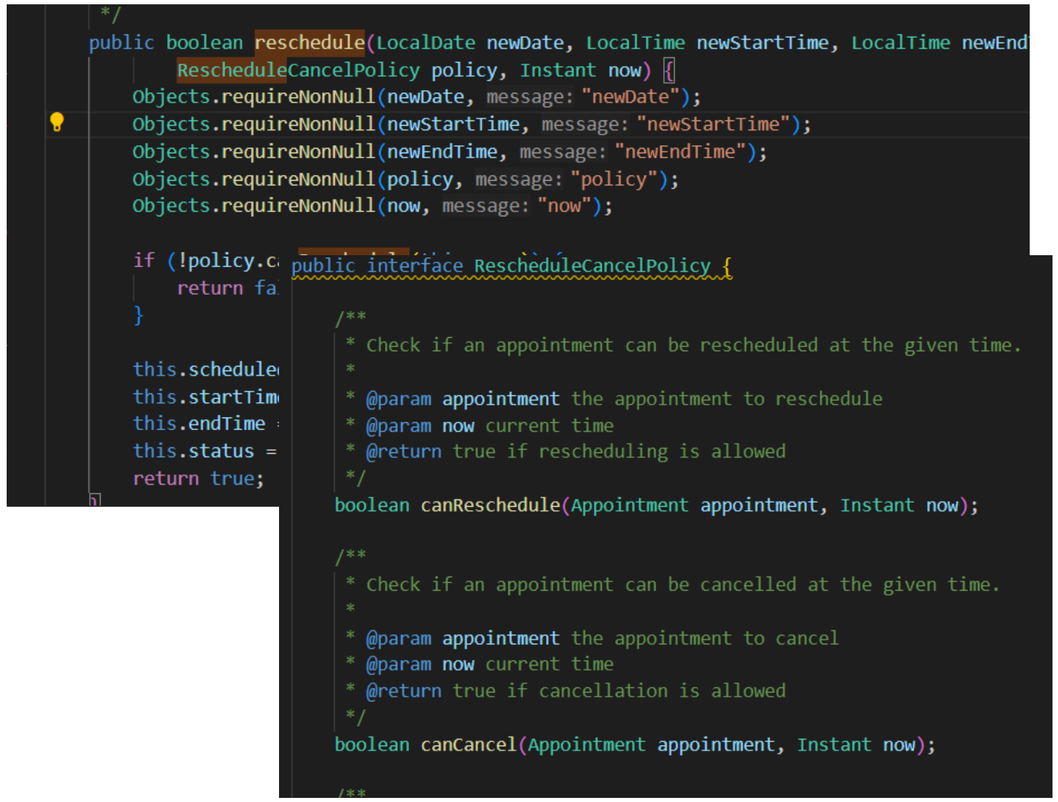
INTERFACE SEGREGRATION PRINCIPLE (ISP)

- JpaRepository<T, ID> interface segregation
  - Each repository extends JpaRepository with specific entity type
  - Generic CRUD methods inherited (findById, save, delete)
  - Custom query methods added per repository needs
  - Clients only depend on methods relevant to their entity



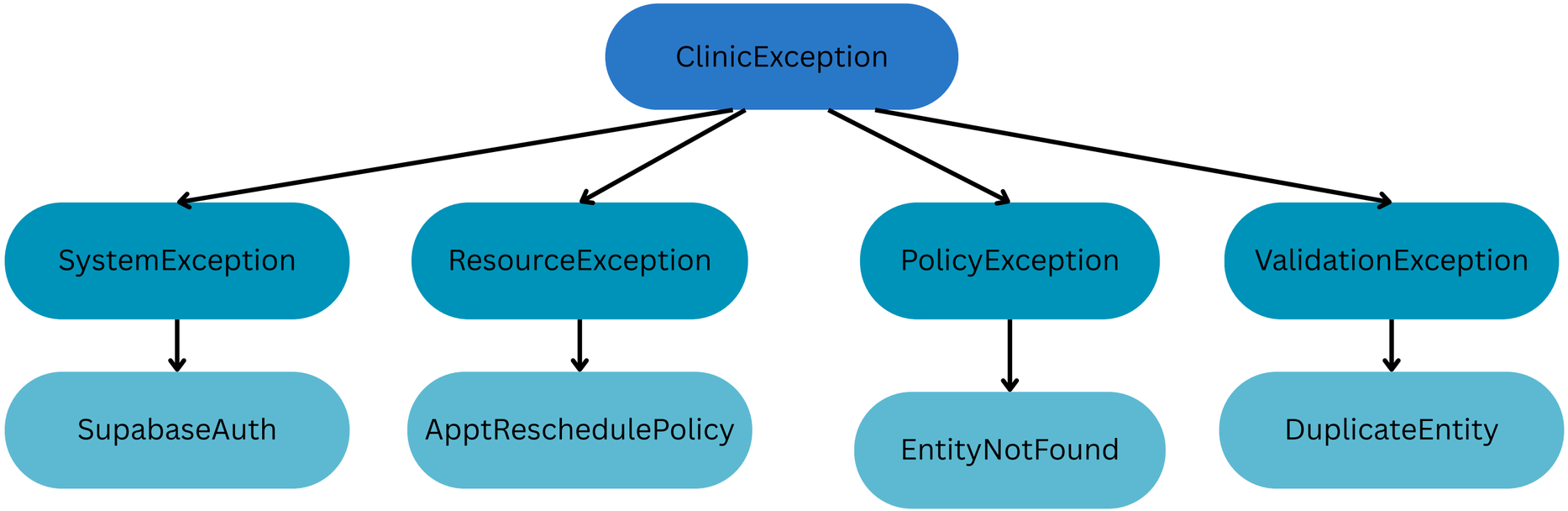
DEPENDENCY INVERSION PRINCIPLE (DIP)

- High-level modules depend on abstractions, not concrete implementations
- Repository Pattern provides full DIP compliance
  - Small, focused interfaces
  - All services depend on repository interfaces (via Spring Data JPA)
  - Database implementation can be swapped without changing services
- Policy Pattern
  - Domain entities (like Appointment) contain business logic that relies on policy interfaces rather than concrete implementations



ADDITIONAL CONSIDERATIONS

1. Exception handling



DESIGN PATTERNS

- Orchestrator Pattern (Service Layer)
  - QueueService orchestrates queue management:
    - Creates queue → Issues tickets → Calls patients → Sends notifications
  - AppointmentService orchestrates booking workflow:
    - Validates entities → Checks rules → Creates appointment → Saves → Sends notification
  - Services coordinate multiple repositories and domain objects

DESIGN PATTERNS

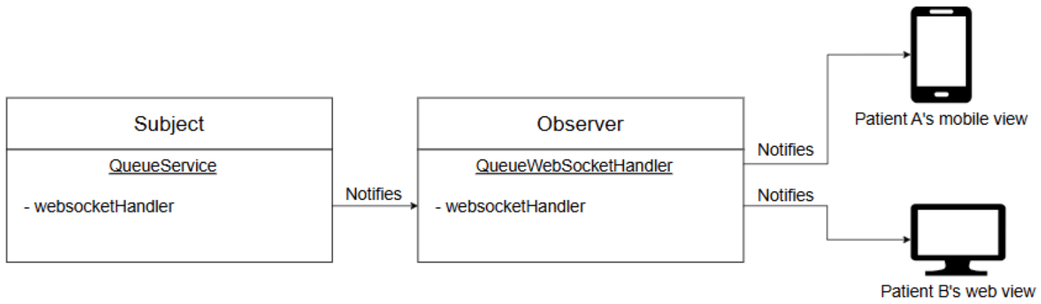
- Static Factory Method Pattern
  - A static method that returns an instance of a class, instead of calling a constructor with new

```
Appointment appt = Appointment.create(patient, doctor, clinic, request.getScheduledDate(),
        request.getStartTime(), request.getEndTime());
```

  - Benefits:
    - Better encapsulation
    - Validation before object creation
    - Single point of change

```
public static Appointment create(Patient patient, Doctor doctor, Clinic clinic,
        LocalDate scheduledDate, LocalTime startTime, LocalTime endTime) {
    Objects.requireNonNull(patient, "patient");
    Objects.requireNonNull(doctor, "doctor");
    Objects.requireNonNull(clinic, "clinic");
    Objects.requireNonNull(scheduledDate, "scheduledDate");
    Objects.requireNonNull(startTime, "startTime");
    Objects.requireNonNull(endTime, "endTime");
```

DESIGN PATTERNS



- Observer Pattern (Websocket)
  - QueueWebSocketHandler maintains list of observers (WebSocket sessions)
  - Notifies all subscribers when queue state changes
  - broadcastQueueUpdate() pushes updates to all connected clients
  - Benefits:
    - Real-time bidirectional communication (not just request-response)
    - Efficient push notifications (no wasteful polling)
    - Loose coupling between services and clients
    - Scalable architecture (one change efficiently notifies multiple observers)

DEMO

LINK TO SUBMISSION

Google Drive Link:

https://drive.google.com/drive/folders/1Y3-fDb3EeQ8vg510Iaw2aI4n7UcVMS7t?usp=drive_link

Includes:

- Code
- Readme
- Sequence diagram
- UML Class diagram